

AI ENGINEERING CHALLENGE

DELIVERABLES

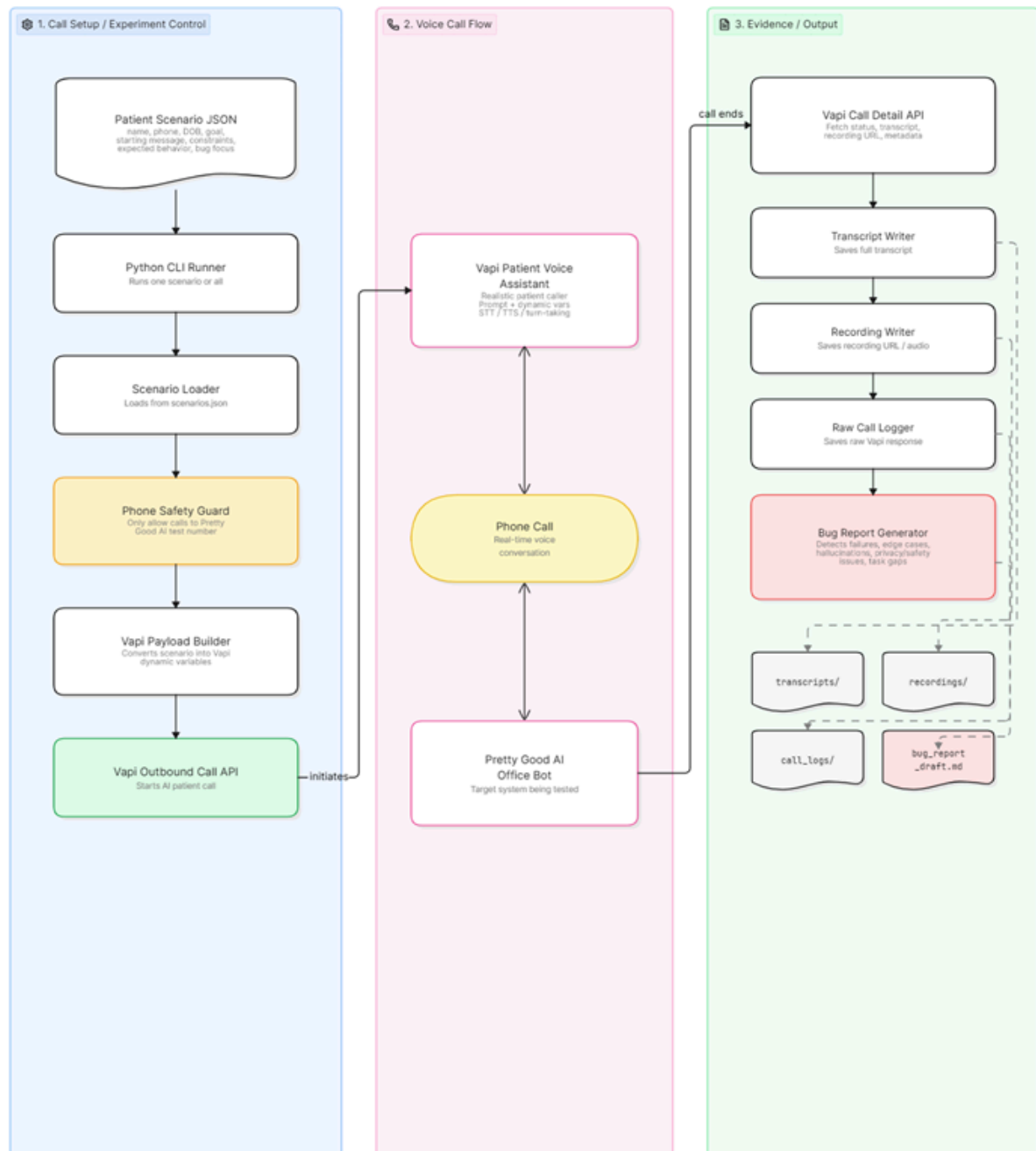
Working Code - [GITHUB](#)

1. The GitHub repo includes the working Python code, README, bug report, transcripts, and call recordings. Audio recordings are included in MP3 format under recordings/mp3/, and transcripts are included under transcripts/.
2. To run the calls, use `python scripts/run_all_scenarios.py --limit 10`, or run one scenario with `python scripts/run_call.py --scenario <scenario_id>`. The bug report is included in `bug_report_draft.md`.
3. The .txt transcript is quick to review, the .mp3 file proves the real conversation, and the .json files preserve call IDs, timestamps, speaker roles, recording URLs, and speech-to-text metadata.

DESIGN DECISIONS : I used Python for the orchestration layer because the main work of the system is scenario management, API integration, safety validation, file writing, and automated testing. Keeping the voice infrastructure in Vapi and the testing logic in Python creates a clean separation of responsibilities: Vapi handles the real-time call experience, while Python controls repeatability, safety, logging, and analysis.

- I chose Vapi because it provides a reliable managed voice infrastructure for outbound AI phone calls, including telephony, speech-to-text, text-to-speech, turn-taking, recordings, and transcript capture. This let the project focus on the evaluation problem: designing realistic patient scenarios, safely executing calls, collecting evidence, and identifying failures in the office bot.
- I designed the system around predefined scenarios, so each call is repeatable and tied to a specific evaluation goal. Each scenario includes patient details, the opening message, expected behavior, and failure conditions, which makes it easier to compare results across calls and support bug reports with concrete evidence.
- I added a strict phone-number guard and made `DRY_RUN=true` the default because this system can trigger real outbound calls. These safeguards reduce the risk of accidental calls, protect against misconfiguration, and allow payloads and scenarios to be tested before any real call is placed.
- I stored outputs as audio, readable transcripts, structured JSON transcripts, and metadata because reviewers need both human-readable evidence and raw technical evidence. The .txt transcript is quick to review, the .wav file proves the real conversation, and the .json files preserve call IDs, timestamps, speaker roles, recording URLs, and speech-to-text metadata.

AI Patient Voice Bot Architecture with Vapi



BUG REPORT : I reviewed the calls based on what would matter in a real patient phone conversation: did the bot finish the task, remember what the patient already said, handle identity details correctly, answer policy questions safely, and give a clear next step when it could not help.

The main problems I found were around memory and follow-through. In one rescheduling call, the bot seemed to ask for information again instead of moving forward with what it already had. In the cancellation call, the patient asked to cancel and also asked about a fee, but the bot never gave a clear final answer. In the identity correction call, the bot did not cleanly retain the corrected date of birth. Even in a test environment, this matters because healthcare workflows depend on accurate identity confirmation. In the insurance call, the bot gave an answer about Aetna PPO, but it was not clear whether that answer came from a verified clinic policy or was just a broad statement. Overall, the bot worked in parts, but it needs better context memory, clearer endings, and more careful policy handling. Every bug is clearly detailed in the repo.